

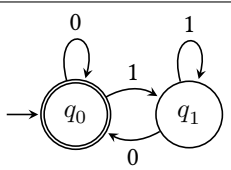
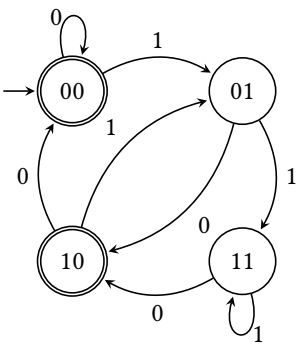
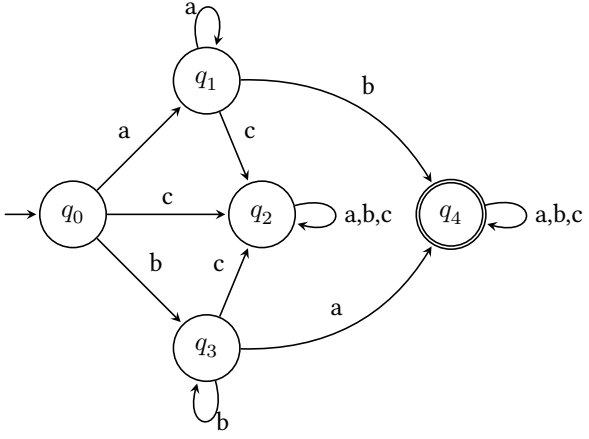
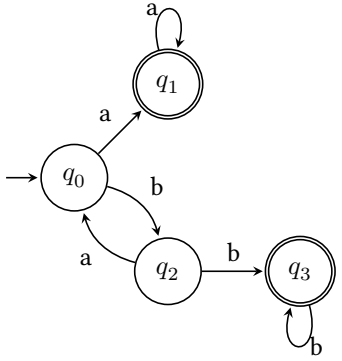
AUTOMATES FINIS

I - Automates finis déterministes

I.1 - Premiers exemples

Les automates constituent un modèle de calcul puissant qui irrigue de nombreuses branches de l'informatique.

Dans l'idée, un automate est une sorte de graphe orienté dont les arêtes sont étiquetées par des lettres, avec boucles et multi-arêtes, muni d'un sommet appelé état initial (et repéré par une flèche entrante), et de sommets appelés états finaux (repérés par des doubles cercles). Il sert à reconnaître des mots : on commence dans l'état initial et pour chaque lettre du mot on suit la flèche étiquetée par cette lettre jusqu'à la fin de lecture du mot. Le mot est accepté si on tombe alors dans un état final. Sinon, il est rejeté.

Automate :	Commentaires :
	Exemples de mot reconnu : Ensemble des mots reconnus :
	Que dire de cet automate par rapport au précédent ?
	Comment caractériser les mots qui nous font passer de l'état q_0 à l'état q_4 ?
	Un automate non complet : si l'on essaie de suivre une flèche inexistante (en lisant b depuis l'état q_1, par exemple), on a perdu. Quels sont les mots acceptés ? En donner une expression régulière.

I.2 - Définitions

Définition 1

Automate fini déterministe :

Si $a \in \Sigma$, on note $q \xrightarrow{a} q'$ pour $\delta(q, a) = q'$. Parfois, on peut aussi trouver la notation $q.a$ pour alléger $\delta(q, a)$, mais je déconseille son utilisation. Elle apporte beaucoup de confusion pour les non-habitués.

Définition 2

Un automate $(\Sigma, Q, q_I, F, \delta)$ est dit **complet** si

Représentation graphique :

On représente un automate sous la forme d'un multi-graphe orienté tel que :

- les sommets sont les états ;
- si $\delta(q, s) = q'$, il y a un arc étiqueté par s de l'état q vers l'état q' ;
- s'il y a plusieurs arcs $q \xrightarrow{a_1} q', \dots, q \xrightarrow{a_n} q'$, on ne représentera graphiquement qu'une arête, que l'on étiquettera $a_1, \dots, a_n$;
- on représente l'état initial avec une flèche entrante ;
- on représente les éventuels états finaux en les entourant deux fois.

Exercice :

1. Dessiner l'automate suivant :

- $\Sigma = \{0, 1\}$
- $Q = \{A, B, C\}$
- $q_I = A$
- $F = \{C\}$

• δ définie par

δ	0	1
A	A	B
B	C	
C	C	

2. Donner Σ, Q, q_I, F et δ pour les deux derniers automates donnés en exemple introductif.

I.3 - Langage d'un automate

Définition 3

Fonction de transition étendue :

Soit $A = (Q, q_I, F, \delta)$ un automate sur l'alphabet Σ . On étend δ en une fonction partielle $\delta^* : Q \times \Sigma^* \rightarrow Q$ comme suit :

Autrement dit, on a $\delta^*(q, a_1 \dots a_n) = q'$ si la lecture successive des lettres $a_1, \dots, a_n$ depuis l'état q nous fait arriver dans l'état q' .

Proposition 1

Quelques propriétés de δ^* :

On a, pour tout état $q \in Q$, pour tous mots $u, v \in \Sigma^*$ et pour toute lettre $a \in \Sigma$:

- $\delta^*(q, a) = \delta(q, a)$
- $\delta^*(q, uv) = \delta^*(\delta^*(q, u), v)$, sous réserve de bonne définition (on lit le mot u puis le mot v).
- $\delta^*(q, au) = \delta^*(\delta(q, a), u)$ (on lit la première lettre séparément au lieu de la dernière).

On pourra noter $q \xrightarrow{u} q'$ pour $\delta^*(q, u) = q'$.

Ainsi $q \xrightarrow{uv} q'$ revient à écrire $q \xrightarrow{u} q'' \xrightarrow{v} q'$ pour un certain état $q'' \in Q$, y compris quand u ou v est vide ou réduit à une lettre.

Démonstration. Sur papier.

□

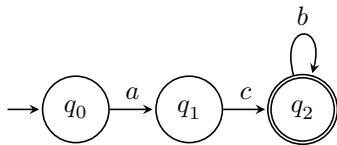
Définition 4

Mot accepté, langage reconnu :

- On dit que l'automate A **reconnaît** (ou **accepte**) le mot u si :
- Le **langage reconnu** par l'automate A , noté $\mathcal{L}(A)$, est l'ensemble des mots reconnus par A .

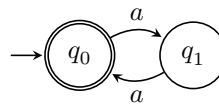
Exercice : Pour chacun des automates suivants, donner une expression régulière décrivant le langage qu'il reconnaît :

L'automate A_1 :



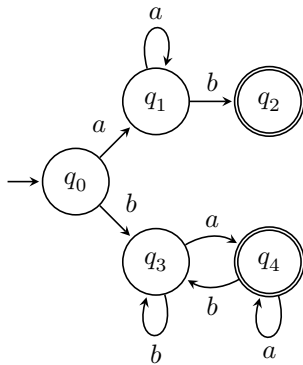
Réponse :

L'automate A_2 :



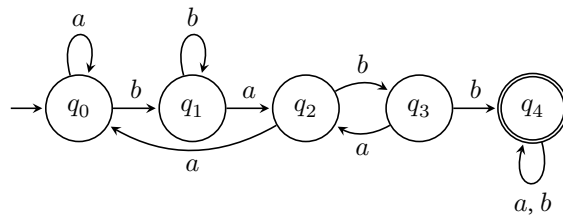
Réponse :

L'automate A_3 :



Réponse à écrire sur papier !

L'automate A_4 :



Réponse à écrire sur papier !

Exercice : Dessiner des automates reconnaissant les langages suivants :

- $\Sigma = \{a, b, c\}$, L décrit par $(a|b)^*$.
- $\Sigma = \{a, b, c\}$, $L = \{u \in \Sigma^*, |u|_a = 1 \pmod 3\}$.
- $\Sigma = \{a, b\}$, $L = \{u \in \Sigma^*, |u|_a \pmod 2 = |u|_b \pmod 3\}$.
- $\Sigma = \{a, b\}$, L décrit par $(a|b)^*a^2(a|b)^*$.

À faire sur feuille !

Définition 5

Deux automates finis (sur un même alphabet) sont dits **équivalents** s'ils reconnaissent le même langage.

Remarque :

- On peut vouloir considérer que deux expressions régulières sont "égales" si elles sont équivalentes, car on s'intéresse au langage qu'elles décrivent (on parle de *sémantique dénotationnelle*), mais ce n'est pas le cas pour deux automates équivalents. On veut aussi prendre en compte le coût du calcul de l'automate, et en particulier son nombre d'états. On s'intéresse à l'évolution de l'état de la machine au cours du calcul (on parle de *sémantique opérationnelle*).

Le lien entre deux automates équivalents est similaire au lien entre deux implémentations d'une même fonction mathématique dans un langage de programmation : elles n'utilisent pas forcément le même algorithme, elles n'ont pas forcément la même complexité...

Proposition 6

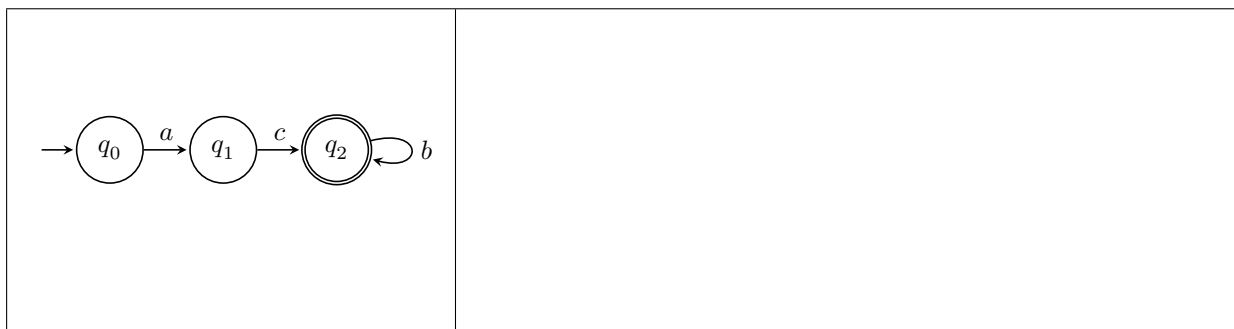
Un **état puits** dans un automate est un état dont on ne peut sortir, i.e. un état q tel que $q \xrightarrow{a} q$ pour tout $a \in \Sigma$.

Pour tout automate fini à n états, il existe un automate complet ayant au plus $n + 1$ états qui lui soit équivalent.

Démonstration. Sur papier !

□

Exemple : Compléter l'automate suivant. (On dessinera à droite un automate complet équivalent.)



Remarque :

- Pour alléger le dessin, on se contentera souvent d'indiquer la présence d'un état puits sans le représenter.

Définition 7

Un langage L sur un alphabet Σ est dit **reconnaisable** s'il existe un automate fini A d'alphabet Σ tel que $\mathcal{L}(A) = L$.

Remarque :

- Le théorème de Kleene affirme que les langages acceptés par les automates sont exactement les langages réguliers. Ce sera un des résultats fondamentaux de ce chapitre !
- Par la proposition qui précède, un langage L est reconnaissable si et seulement si il existe un automate fini A **complet** tel que $\mathcal{L}(A) = L$. On utilisera souvent un automate complet dans les preuves.

I.4 - Automate produit

On va maintenant voir une construction d'automate qui permet de simuler en parallèle l'exécution de deux automates $\mathcal{A}$ et $\mathcal{A}'$ donnés.

⚠ Attention ! Cette section est à copier sur feuille depuis le tableau. Ceci n'est qu'une annexe de la section. **⚠**

Remarque (automate produit) :

- L'hypothèse de complétude sur A et A' n'est pas fondamentale, elle ne fait que simplifier un peu la définition, et assurer que l'automate produit est également complet.

I.5 - Automate émondé

Définition 8

États accessibles, états co-accessibles :

Soit $A = (\Sigma, Q, q_I, F, \delta)$ un automate fini déterministe. Un état q de A est dit :

- **accessible** si :
- **co-accessible** si :

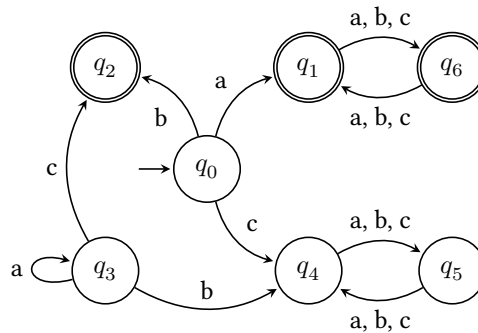
Remarques :

- Autrement dit, un état q est accessible s'il existe un chemin de q_I vers q dans le graphe sous-jacent de l'automate (le graphe obtenu en oubliant les étiquettes des transitions).
- De même, q est co-accessible s'il existe un chemin de q vers l'un des états finaux. Ou encore de l'un des états finaux vers q dans le graphe sous-jacent renversé (celui dans lequel on a inversé le sens de toutes les flèches).

Comment trouver l'ensemble des états accessibles dans un graphe ? Des états co-accessibles ?

Exemple :

Dans l'automate ci-dessous, entourez en vert les états qui ne sont *pas* accessibles, et en rouge les états qui ne sont *pas* co-accessibles.



On remarque que les états qui ne sont pas accessibles ou pas co-accessibles sont "inutiles" dans le sens où ils ne peuvent pas participer à former un mot reconnu par l'automate. On peut donc tous les supprimer (on parle alors d'émonder l'automate) ou bien les remplacer par un unique état puits "poubelle", sans changer le langage reconnu par l'automate.

Proposition 3 Définition 9

Automate émondé :

Émondage :

Soit A un AFD. L'automate obtenu en ne gardant que les états simultanément accessibles et co-accessibles de A , et en restreignant δ en conséquence, est un automate émondé équivalent à A .

Remarques :

- Essentiellement, émonder un automate, c'est supprimer les états qui ne servent visiblement à rien.
- En général, un automate émondé n'est pas complet, et on ne peut pas le compléter en gardant le caractère émondé. Il faut alors s'autoriser un (unique) état non co-accessible : l'état puits.
- ⚠ Ce n'est pas parce qu'un automate est émondé qu'il est minimal : il peut tout-à-fait y avoir un autre automate, de forme a priori différente, qui reconnaît le même langage en utilisant moins d'états. Nous reviendrons plus tard sur le problème de la minimisation d'un automate déterministe.

Exercice :

Émonder l'automate exemple précédent et montrer que l'automate obtenu n'est pas minimal.

À faire sur feuille !

II - Automates finis non déterministes

II.1 - Premiers exemples

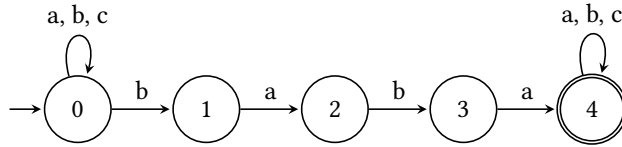
Idée générale :

Les automates finis non déterministes sont une extension des automates finis déterministes qui autorise, en plus :

- plusieurs états initiaux (on remplace q_I par un ensemble $I \subseteq Q$)
- plusieurs transitions partant d'un même état et étiquetées par la même lettre (δ est une relation et pas nécessairement une application partielle).

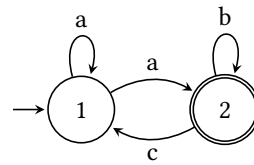
Exemples :

Automate "baba" :



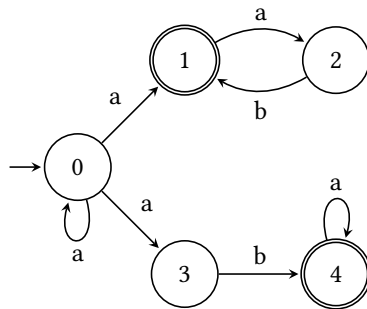
Commentaires :

Automate B_1 :



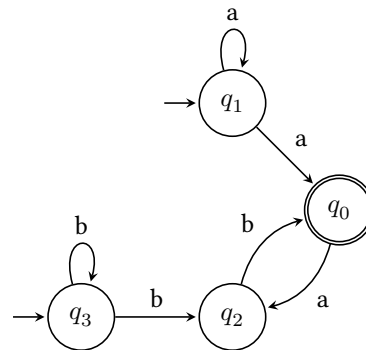
Commentaires :

Automate B_2 :



Commentaires :

L'automate B_3 :



Commentaires :

II.2 - Définitions

Automate fini non déterministe :

Un **automate fini** (non déterministe), ou **NFA** ou **AFND**, est un quintuplet $A = (\Sigma, Q, I, F, \delta)$ où :

- Σ est un alphabet (fini)
- Q est un ensemble fini d'**états**
- I
- $F \subseteq Q$ est l'ensemble des **états finaux** ou **états acceptants** ;
- δ

On pourra noter $q' \in \delta(q, a)$ ou $q \xrightarrow{a} q'$ pour $(q, a, q') \in \delta$.

Remarques :

- Les automates finis déterministes peuvent bien être considérés comme des cas particuliers d'automates finis non déterministes. Avec les notations de la définition, A est déterministe si :

- $|I| = 1$
- la relation δ est fonctionnelle : $(q \xrightarrow{a} q' \text{ et } q \xrightarrow{a} q'' \Rightarrow q' = q'')$.

Fonction de transition étendue :

On étend la fonction de transition δ aux parties de Q en posant, pour $X \subseteq Q$:

$$\delta(X, a) =$$

On définit ensuite δ^* :

On confond souvent un état $q \in Q$ avec le singleton $\{q\} \in \mathcal{P}(Q)$ lorsque le contexte est clair. Autrement dit, $\delta^*(q, w) = \delta^*(\{q\}, w)$ est l'ensemble des états que l'on peut atteindre à partir de q en suivant des transitions étiquetées par les lettres de w (dans l'ordre).

Remarques :

- Dans le cas où A se trouve être déterministe (I est un singleton, δ est une relation fonctionnelle), la fonction δ^* , la notion de mot accepté et la notion de langage reconnu coïncident bien avec celles définies pour les automates déterministes.
- À nouveau, on montre aisément les points suivants :
 - $\delta^*(q, a) = \delta(q, a)$
 - $\delta^*(q, uv) = \delta^*(\delta^*(q, u), v)$ (par récurrence sur $|v|$ avec u fixé).
 - $\delta^*(q, au) = \delta^*(\delta(q, a), v)$

Mot accepté, langage reconnu :

- Un mot w est **accepté** par A si :

Autrement dit, si :

- Le **langage reconnu** par A est l'ensemble des mots reconnus par A . On le notera $\mathcal{L}(A)$.

Exercice : Déterminer les langages reconnus par les automates non déterministes donnés en exemple introductif. Compléter les commentaires des exemples.

Exercice :

Construire des automates finis (non déterministes) reconnaissant les langages suivants :

1. les mots sur $\Sigma = \{a, b\}$ ayant bba comme suffixe ;
2. les mots w sur $\Sigma = \{a\}$ vérifiant $|w| = 1 \pmod 3$ ou $|w| = 2 \pmod 5$;
3. les mots sur $\Sigma = \{a, b, c\}$ ayant bba ou cab comme facteur.

À faire sur feuille !

II.3 - Détermination

Les automates non déterministes sont plus faciles à écrire, à concevoir et à manipuler, mais sont-ils "équivalents" aux automates déterministes, ou sont-ils "plus puissants" strictement ?

On va maintenant voir qu'ils permettent de décrire les mêmes langages que les automates déterministes.

Idée générale de la transformation :

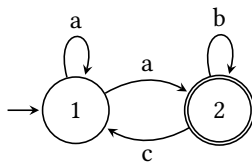
Dans le cas d'un automate déterministe (complet, pour simplifier), on sait précisément dans quel état on se trouve après avoir lu un mot w depuis l'état initial. Pour un automate non déterministe, l'état atteint est le résultat d'une série de choix : le choix de l'état initial, et le choix de la transition suivie à chaque étape. Ce qu'on peut déterminer entièrement

pour un automate non déterministe, c'est l'ensemble des états dans lesquels on peut se trouver après la lecture d'un mot w . Si l'on part d'un état q et que l'on lit u , les états possibles sont précisément ceux de $\delta^*(q, u)$.

Exemple :

Reprenons l'automate B_1 :

Dessiner un automate déterministe équivalent à B_1 , qu'on appellera *automate des parties* :



Automate des parties :

Définition 13

Automate des parties :

Soit $A = (\Sigma, Q, I, F, \delta)$ un automate (non-déterministe). L'**automate des parties** associé à A est l'automate déterministe complet $(\Sigma, Q', q'_I, F', \eta)$ où :

Remarques :

- Dans le dernier point, $\delta(X, a) := \bigcup_{q \in X} \delta(q, a)$ est l'extension de la fonction de transition non déterministe de A aux parties de Q . En revanche, η est une fonction de transition déterministe : à un état de A' (c'est-à-dire un ensemble d'états de A) et une lettre, elle associe un unique état de A' (c'est-à-dire à nouveau un unique ensemble d'états de A).
- L'automate des parties est complet par construction : l'état $\emptyset$ joue le rôle d'état puits. Quand on représentera l'automate, on l'omettra presque systématiquement pour alléger.

Combien d'états l'automate des parties possède-t-il au total ?

En pratique, on ne construira que les états *accessibles*, et le nombre d'états sera le plus souvent nettement inférieur à cette borne. Mais cette borne peut être (presque) atteinte, comme nous le verrons en exercice.

Soit A un automate fini et A' l'automate des parties associé. On a $\mathcal{L}(A) = \mathcal{L}(A')$.

En particulier, on en déduit le théorème suivant :

Un langage est reconnaissable par un automate fini déterministe si et seulement si il est reconnaissable par un automate fini non déterministe.

Théorème 1 Proposition 4

Démonstration. Sur feuille!

□

Remarques :

- On pourra donc simplement parler d'un « langage reconnaissable », sans préciser par quel type d'automate fini.
- Le non déterminisme n'apporte donc pas de "puissance" supplémentaire. Cependant, il existe des langages pour laquelle la reconnaissance par un automate déterministe est coûteuse : ils sont reconnaissables par un automate non-déterministe à n états, mais le nombre minimal d'états d'un automate déterministe les reconnaissant est de l'ordre de 2^n .

Procédure de déterminisation :

En pratique, on ne va construire que les états accessibles de l'automate des parties. On effectue un parcours de l'automate, tout en construisant l'automate des parties en parallèle. Dans le pseudo-code qui suit, on effectue un parcours en largeur, mais cela n'a pas vraiment d'importance.

Pseudo-code

```
1  Entrée : un automate  $A = (\Sigma, Q, I, F, \delta)$ .
2  Sortie : un automate déterministe  $A' = (\Sigma, Q', q'_I, F', \delta')$  tel que  $\mathcal{L}(A) = \mathcal{L}(A')$ .
3  CONSTRUCTION :
4      // Initialisation :
5       $Q' \leftarrow \{I\}$ 
6       $q'_I \leftarrow I$ 
7       $\delta' \leftarrow \emptyset$ 
8      ouverts  $\leftarrow \{I\}$ 
9      //parcours :
10     TantQue ouverts  $\neq \emptyset$  Faire :
11         nouveaux  $\leftarrow \emptyset$ 
12         Pour  $X \in$  ouverts Faire :
13             Pour  $a \in \Sigma$  Faire :
14                  $Y = \bigcup_{q \in X} \delta(q, a)$ 
15                 Si  $Y \not\subseteq Q'$  Alors :
16                      $Q' \leftarrow Q' \cup \{Y\}$ 
17                     nouveaux  $\leftarrow$  nouveaux  $\cup \{Y\}$ 
18                  $\delta' \leftarrow \delta' \cup \{(X, a, Y)\}$ 
19             ouverts  $\leftarrow$  nouveaux
20      $F' \leftarrow \{X \in Q', X \cap F \neq \emptyset\}$ 
```

Remarques :

- Le pseudo-code est un peu vague, surtout en ce qui concerne les structures de données à utiliser. On ne vous demande pas de savoir programmer cet algorithme sans aide. En revanche, il faut l'avoir compris et savoir l'appliquer à la main sur un exemple de taille raisonnable.
- Par construction, tous les états de l'automate A' sont accessibles. Il n'y a en revanche aucune raison qu'ils soient co-accessibles.

Exercice :

Déterminiser les automates B_2 et B_3 donnés en exemples introductifs.

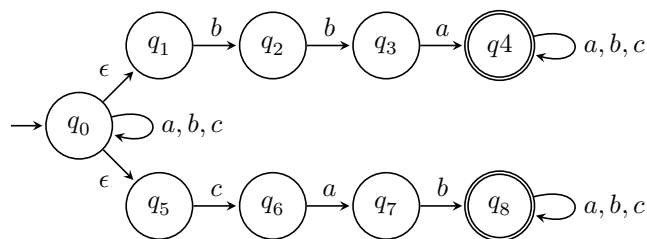
À faire sur feuille

II.4 - ϵ -transitions

Principe général :

Exemple :

Pour construire un automate reconnaissant l'ensemble des mots qui contiennent bba ou cab en facteur, c'est à dire le langage décrit par $\Sigma^*(bba|cab)\Sigma^*$, on peut utiliser des ϵ -transitions de la manière suivante :



Complétons notre définition d'automates finis non déterministes pour y ajouter les ϵ -transition, de la manière suivante :

Un **automate fini (non déterministe) avec ϵ -transitions** (ou **transitions spontanées**), ou **ϵ -NFA** ou **ϵ -AFND** est un quintuplet $A = (\Sigma, Q, I, F, \delta)$ où :

- Σ est un alphabet (fini);
- Q est un ensemble fini d'états;
- $I \subseteq Q, I \neq \emptyset$ est l'ensemble des **états initiaux**;
- $F \subseteq Q$ est l'ensemble des **états finaux** ou **états acceptants**;
-

On pourra noter $q' \in \delta(q, a)$ ou $q \xrightarrow{a} q'$ pour $(q, a, q') \in \delta$.

Là encore, ajouter des ϵ -transitions à nos automates ne change pas les langages qu'on peut reconnaître. On va montrer que pour tout ϵ -NFA, il existe un automate équivalent sans ϵ -transition.

ϵ -fermeture :

Soit $A = (\Sigma, Q, I, F, \delta)$ un ϵ -AFND. Pour un état $q \in Q$, l' ϵ -fermeture $E(q)$ de q est l'ensemble des états atteignables depuis l'état q par une suite (éventuellement vide) d' ϵ -transitions.

Autrement dit, $E(q)$ est la partie de Q définie inductivement par :

On étend cette notion aux parties de Q en posant pour $S \subseteq Q$:

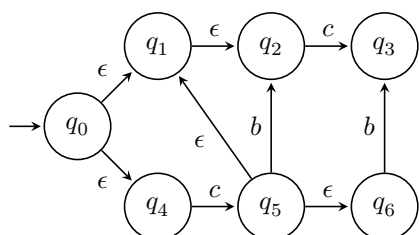
De manière équivalente, on a pour toute partie $S \subseteq Q$:

$$E(S) := \{z \in S \mid \exists q_1 \in S, \exists q_2, q_3, \dots, q_k = z \text{ tels que } \forall i, q_i \in \delta(q_{i-1}, \epsilon)\}$$

Remarques :

- Comme toujours pour une définition d'ensemble inductif, $E(q)$ est le plus petit ensemble contenant q et clos par ϵ -transition.
- Une autre manière de dire la même chose est que $q' \in E(q)$ si et seulement si q' est accessible depuis q dans le graphe orienté obtenu en ne conservant que les ϵ -transitions de A .
- On a $E(E(S)) = E(S)$ pour toute partie S .
- L' ϵ -fermeture est un cas particulier de la notion générale de fermeture réflexive transitive.

Exercice : Déterminer l' ϵ -fermeture de chacun des états de l'automate suivant :



Réponse :

Pour définir le langage reconnu par un ϵ -AFND, il faut définir l'ensemble des états que l'on peut atteindre à partir d'un état donné en suivant des transitions dont la concaténation des étiquettes donne un certain mot. L'idée est que, lorsque l'on concatène les étiquettes des transitions, les ϵ -transitions sont complètement ignorées. Par exemple, un chemin $q_0 \xrightarrow{\epsilon} q_1 \xrightarrow{\epsilon} q_2 \xrightarrow{a} q_3 \xrightarrow{\epsilon} q_4 \xrightarrow{b} q_5 \xrightarrow{a} q_6 \xrightarrow{\epsilon} q_7$ est un chemin étiqueté par aba de q_0 à q_7 .

Fonction de transition étendue (avec ϵ -transitions) :

Pour un ϵ -AFND $A = (\Sigma, Q, I, F, \delta)$, on définit la fonction de transition étendue δ^* par :

Remarques :

- On a aussi, de manière équivalente, $\delta^*(q, wa) = \bigcup_{q' \in \delta^*(q, w)} E(\delta(q', a))$.
- On peut à nouveau étendre la définition de δ et δ^* aux parties X de Q , en posant :
 - $\delta(X, a) = \bigcup_{q \in X} \delta(q, a)$;
 - $\delta^*(X, u) = \bigcup_{q \in X} \delta^*(q, u)$.

On a alors :

$$\delta^*(X, \epsilon) = E(X) \text{ et } \delta^*(X, wa) = E(\delta^*(X, w), a).$$

Exercice :

Déterminer $\delta^*(q_0, \epsilon)$, $\delta^*(q_0, b)$, $\delta^*(q_0, c)$, $\delta^*(q_0, bc)$ et $\delta^*(q_0, cb)$ pour l'automate de l'exercice précédent.
À faire sur feuille!

Langage d'un ϵ -AFND :

Un mot $u \in \Sigma^*$ est **reconnu** par un ϵ -AFND $A = (\Sigma, Q, I, F, \delta)$ si :

Comme pour les autres types d'automates finis, on note $\mathcal{L}(A)$ le langage reconnu par A .

Tout ϵ -AFND est équivalent à un AFND (sans ϵ -transitions) possédant le même nombre d'états.

Démonstration. Sur papier!

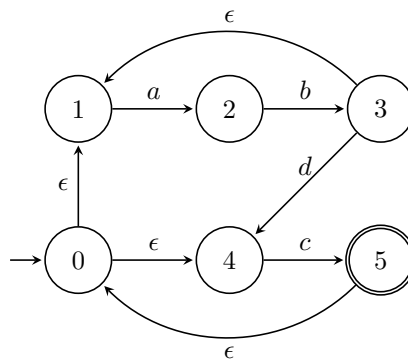
□

Remarques :

- Les langages reconnaissables par un ϵ -AFND sont donc exactement les mêmes que ceux reconnaissables par un AFD : il suffit d'éliminer les ϵ -transitions puis d'utiliser la procédure de déterminisation. Encore une fois, on parle simplement de "langage reconnaissable" sans préciser par quel type d'automate fini.
- Il est tout à fait possible de réaliser ces deux étapes en même temps (passer directement d'un ϵ -AFND à un AFD).

Exercice :

Éliminer les ϵ -transitions de l'automate ci-dessous :



À faire sur feuille!

Soit A un ϵ -AFND. Il existe un ϵ -AFND A' équivalent n'ayant qu'un seul état initial, sans transition entrante.

Démonstration. Sur papier ! □

III - Langages reconnaissables

III.1 - Théorème de Kleene : sens direct

Dans cette partie, nous allons montrer que les langages rationnels sont reconnaissables par un automate fini, ce qui constitue le sens direct du théorème de Kleene (les langages reconnaissables sont exactement les langages réguliers).

⚠ Attention! Cette section est à copier sur feuille depuis le tableau. Ceci n'est qu'une annexe de la section. **⚠**

Remarque :

- On a montré dans la partie sur les automates déterministes que les langages reconnaissables sont clos par union¹ (et par intersection). Il ne reste donc plus qu'à montrer que les langages reconnaissables contiennent $\mathcal{L}(\emptyset)$, $\mathcal{L}(\epsilon)$ et $\mathcal{L}(a)$ pour tout $a \in \Sigma$, et qu'ils sont clos par concaténation et par étoile de Kleene, pour conclure que tous les langages réguliers sont reconnaissables.

Exercice :

1. Construire l'automate de Thompson associé à l'expression $(ab|a)^*c^*$.
2. Éliminer les ϵ -transitions de cet automate et élaguer l'automate obtenu.

III.2 - Théorème de Kleene : sens indirect

On veut maintenant montrer que les langages reconnaissables sont rationnels. Les preuves de ce résultat fournissent également des algorithmes pour obtenir une expression régulière décrivant le langage d'un automate A donné.

Remarque :

- Il est beaucoup plus rarement utile de passer d'un automate à une expression que de faire l'inverse.
- Les expressions obtenues sont en général extrêmement lourdes, même quand le langage peut être décrit par une expression simple.

III.2.i - Méthode de McNaughton et Yamada

Cette première méthode fournit la preuve la plus simple au sens indirect, et est la plus simple à programmer, mais elle est assez pénible à utiliser « à la main »

Idée générale :

On procède comme l'algorithme de Floyd-Warshall pour le calcul des distances minimales dans un graphe pondéré. On numérote les états de l'automate de 0 à $n - 1$.

Soit A un automate à n états (qui peut a priori être non déterministe et posséder des ϵ -transitions, cela ne changera rien). On numérote les états de A de 0 à $n - 1$.

Notons $L_{i,j}^{(k)}$ l'ensemble des mots qu'on peut lire en suivant un chemin de l'état i à l'état j dont tous

1. On peut en fait le montrer de manière bien plus simple avec des automates non déterministes : cf automates de Thompson

les sommets intermédiaires sont strictement inférieurs à k , i.e. :

$$L_{i,j}^{(k)} = \{w \in \Sigma^* \mid \delta^*(i, w) = j \text{ et } \forall w' \notin \{\epsilon, w\} \text{ préfixe de } w, \delta^*(i, w') < k\}.$$

$$\text{Alors on a pour tout } k \in \llbracket 0; n-1 \rrbracket, \boxed{L_{i,j}^{(k+1)} = L_{i,j}^{(k)} \cup \left(L_{i,k}^{(k)} \cdot \left(L_{k,k}^{(k)} \right)^* \cdot L_{k,j}^{(k)} \right)}$$

Démonstration. Sur papier ! □

On peut donc construire successivement des expressions régulières $R_{i,j}^{(k)}$ reconnaissant les langages $L_{i,j}^{(k)}$. On calcule une suite de matrices $(R^{(k)})_{0 \leq k \leq n}$. On en déduit un algorithme pour construire une expression régulière dénotant le langage reconnu par l'automate, grâce à $R^{(n)}$:

ALGORITHME DE McNAUGHTON ET YAMADA (sur un automate A) :

Initialisation :

$L_{i,j}^{(0)}$ est l'ensemble des étiquettes des chemins reliant i à j sans état intermédiaire, c'est-à-dire en utilisant une unique transition, ou éventuellement aucune transition si $i = j$. Notons $\alpha(i, j) = \{a \in \Sigma \mid q_j \in \delta(q_i, a)\}$ l'ensemble des lettres étiquetant une transition de i vers j . On peut alors prendre :

$$R_{i,i}^{(0)} = \begin{cases} \epsilon & \text{si } \alpha(i, i) = \emptyset \\ \epsilon | a_1 | \dots | a_p & \text{si } \alpha(i, i) = \{a_1, \dots, a_p\} \end{cases}$$

$$R_{i,j}^{(0)} = \begin{cases} \emptyset & \text{si } \alpha(i, j) = \emptyset \\ a_1 | \dots | a_p & \text{si } \alpha(i, j) = \{a_1, \dots, a_p\} \end{cases}$$

Boucle :

Pour k variant de 1 à n , on calcule (grâce à la formule de récurrence donnée précédemment) :

$$R_{i,j}^{(k+1)} = R_{i,j}^{(k)} \mid R_{i,k}^{(k)} \left(R_{k,k}^{(k)} \right)^* R_{k,j}^{(k)}$$

Résultat :

La matrice $R^{(n)}$ donne alors, pour chaque couple (i, j) de sommets, une expression décrivant le langage $L_{i,j} = \{w \in \Sigma^* \mid q_j \in \delta^*(q_i, w)\}$.

Pour obtenir une expression décrivant le langage reconnu par l'automate, il suffit donc de prendre l'union de tous les $R_{i,j}^{(n)}$ pour $q_i \in I$ et $q_j \in F$.

Par exemple, dans le cas le plus fréquent, s'il n'y a qu'un seul état initial, et que de plus on attribue à cet état initial l'indice 0, l'expression finale est $R_{0,f_1}^{(n)} \mid \dots \mid R_{0,f_p}^{(n)}$ où $F = \{f_1, \dots, f_p\}$.

Remarque :

- Si l'on n'effectue aucune simplification dans les expressions intermédiaires, leur taille explose très rapidement : chaque « coefficient » de $R^{(k+1)}$ est obtenu en combinant quatre « coefficients » de $R^{(k)}$, et l'on peut donc raisonnablement penser que les expressions présentes dans la matrice $R^{(n)}$ auront une taille de l'ordre de 4^n ...

Théorème de Kleene :

Les langages reconnaissables sont exactement les langages rationnels.

Autrement dit, pour tout alphabet Σ et tout langage L sur Σ , il existe une expression régulière e telle que $L = \mathcal{L}(e)$ si et seulement si il existe un automate fini déterministe A tel que $L = \mathcal{L}(A)$.

Remarque :

- On a donc L reconnaissable par un ϵ -AFND si et seulement si L reconnaissable par un AFND si et seulement si L reconnaissable par un automate fini déterministe si et seulement si L rationnel. Si cela nous arrange (ce sera souvent le cas), on peut d'ailleurs choisir un automate déterministe complet ou émondé.

III.2.ii - Élimination des états

L'algorithme au programme pour passer d'un automate à une expression régulière est celui de McCluskey et Brzozowski, dit d'**élimination des états**.

Idée générale :

L'algorithme est le suivant :

ÉLIMINATION DES ÉTATS (sur un automate A) :

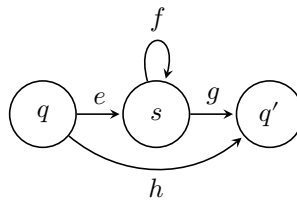
- 1) **On se ramène à un automate avec un seul état initial sans transition entrante, et un seul état final sans transition sortante.** Si ce n'est pas déjà le cas, il suffit de rajouter un unique état initial q_I (respectivement un unique état final q_F) et des ϵ -transition de q_I vers chaque ancien état initial, de l'automate d'origine (respectivement de chaque ancien état final vers q_F).

Schéma :

- 2) **On se ramène au cas où il y a exactement une transition entre chaque paire d'états** (sauf vers l'état initial et depuis l'état final).
 - Si on a deux transitions (q, a, q') et (q, b, q') , on la remplace par une unique transition $(q, a|b, q')$. On répète le processus jusqu'à ce qu'il ne reste plus qu'une transition entre q et q' : l'expression régulière de la transition comporte autant d'unions qu'il y avait de lettres permettant de passer de q à q' .
 - S'il n'y a aucune transition entre deux états q et q' , on peut ajouter la transition $q \xrightarrow{\emptyset} q'$. (En pratique on ne le fera pas, mais cela simplifie la présentation de la méthode.)

3) Tant que l'automate possède un état s qui n'est ni l'état initial ni l'état final : on élimine l'état s .

Pour chaque couple (q, q') où q est un prédécesseur de s et q' un successeur, on a localement la situation suivante :



On remplace la transition h par $h|ef^*g$. Ainsi les transitions e et g deviennent inutiles pour passer de q à q' .

Une fois que l'on a fait cette modification pour tous les couples (q, q') , on peut donc éliminer l'état s (et toutes les transitions depuis et vers s).

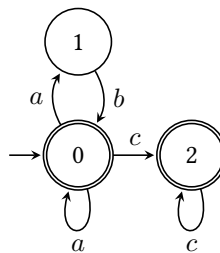
Remarque :

q et q' peuvent être le même état, auquel cas la transition h est une boucle.

4) Lorsqu'il ne reste plus que l'état initial q_I et l'état final q_F , avec une unique transition $q_I \xrightarrow{e} q_F$, alors le langage de l'automate est décrit par e .

Exemple :

Faire tourner l'algorithme d'élimination des états sur l'automate suivant :



À faire sur feuille !

III.3 - Conséquences du théorème de Kleene

Le théorème de Kleene nous assure donc que les langages rationnels sont exactement les langages reconnaissables. Certaines propriétés sont très faciles à prouver pour les langages reconnaissables (via des automates), d'autres pour les langages rationnels (via des expressions régulières). Il est donc assez courant de faire des aller-retours entre les deux modèles.

Proposition 7

L'ensemble des langages reconnaissables est clos par passage au complémentaire :

Si $L \subseteq \Sigma^*$ est reconnaissable, alors $L^c = \{u \in \Sigma^*, u \notin L\}$ est reconnaissable.

Démonstration. Sur papier !

□

Exercice :

1. Montrer que l'ensemble des mots sur $\{a, b\}$ n'admettant pas $abab$ comme facteur est rationnel.
2. Si vous n'avez rien de mieux à faire : essayer de le prouver directement (avec des expressions régulières).

À faire sur papier !

On déduit aisément du théorème de Kleene et des propriétés précédentes les résultats suivantes :

Proposition 8

- Les langages rationnels sont stables par passage au complémentaire. Autrement dit, si $L \subset \Sigma^*$ est rationnel, alors $\Sigma^* \setminus L$ l'est également.
- Les langages reconnaissables sont stables par union.
- Les langages rationnels (et reconnaissables) sont stables par intersection.

Démonstration.

- On vient de montrer que les langages reconnaissables sont stables par complémentaire.
⚠ Il faut faire attention à partir d'un automate fini **déterministe** pour que la construction soit valide.
- Les langages reconnaissables sont les langages rationnels, qui sont stables par union (par construction).
- On a montré que les langages reconnaissables sont stables par intersections et par union (et **il faut savoir le démontrer directement via l'automate produit**!). Mais ici il suffit de remarquer que si L_1 et L_2 sont rationnels, alors d'après les deux propriétés précédentes, il en est de même pour $(L_1^c \cup L_2^c)^c = L_1 \cap L_2$. □

Remarque :

- D'autres propriétés de stabilité des langages rationnels seront prouvées en TD, notamment à l'aide de l'automate produit.

Comme on vient de prouver que les langages rationnels sont clos par intersection et passage au complémentaire, on sait qu'étant données deux expressions régulières e_1 et e_2 , il existe une expression régulière e' telle que $L(e') = L(e_1) \cap L(e_2)$ (et une expression pour $\Sigma^* \setminus L(e_1)$). On est même capable de trouver effectivement une telle expression : il « suffit » de construire un automate pour chaque expression, puis un automate pour l'intersection pour enfin en extraire une expression régulière. Cependant, ce processus est coûteux, surtout que l'expression obtenue peut être beaucoup plus complexe que celles dont on est parti.

Définition 18

On peut donc décider d'enrichir la syntaxe des expressions régulières pour rajouter des opérateurs « $\setminus$ » et « $\cap$ ». On parle alors d'**expressions régulières étendues**.

Remarques :

- ⚠ Ne pas confondre avec les "expressions étendues" dont on parle en informatique "pratique", qui augmentent la puissance du modèle. Par exemple, $(.+)\setminus 1$ reconnaît le langage $\{u^2 \mid u \in \Sigma^+\}$, qui n'est pas rationnel (on le montrera). Ce n'est pas le cas ici, on reconnaît bien toujours exactement les langages rationnels. (Et inversement, aucune syntaxe pratique pour les regex ne contient d'opérateurs généraux pour la différence et l'intersection.)

III.4 - Lemme de l'étoile

⚠ **Attention!** Cette section est à copier sur feuille depuis le tableau. Ceci n'est qu'une annexe de la section. ⚠

Exercice introductif :

Montrer que le langage $L = \{a^n b^n \mid n \geq 0\}$ n'est pas reconnaissable.

Remarques (lemme de l'étoile) :

- En anglais, ce théorème est connu sous le nom de **pumping lemma**, et on trouve parfois l'appellation **lemme de pompage** en français. n est parfois appelée **longueur de pompage** pour L (cette longueur n'est pas unique, on peut toujours la remplacer par une autre longueur plus grande).
- On peut trouver un x , un y et un z pour chaque u assez grand, mais ⚠ il ne s'agira pas des mêmes x , y et z pour tous les u . Attention à l'**ordre des quantificateurs** !
- La réciproque est fausse. Nous le verrons en exercice.
- Le résultat reste en fait valable si l'automate contient des ϵ -transitions mais il faut adapter la preuve, et cela n'a pas vraiment d'intérêt en pratique.

- Fondamentalement, l'idée est que si un automate contient un chemin acceptant de longueur supérieure ou égale à $|Q|$, alors il contient un cycle accessible et co-accessible, qui permet de "pomper".
- Les automates ne contenant *pas* de chemin acceptant de longueur supérieure ou égale à $|Q|$ sont exactement ceux qui reconnaissent un langage de cardinal fini.

En pratique, on utilise surtout le lemme de l'étoile pour montrer qu'un langage n'est pas rationnel, par l'absurde.

Exercice d'application : En utilisant le lemme de l'étoile, montrer que les langages suivants ne sont pas rationnels :

1. $L = \{a^{i^2} \mid i \in \mathbb{N}\}$
2. $L = \{u \in \{a, b\}^* \mid |u|_a = |u|_b\}$

Point méthodologie :

Pour montrer qu'un langage est reconnaissable, on pourra :

- soit **exhiber directement un automate** le reconnaissant
- soit utiliser des **propriétés de clôture**.

Pour montrer qu'un langage n'est *pas* reconnaissable, une technique usuelle est de montrer que **sa reconnaissance nécessiterait un nombre infini d'états** : c'est ce qu'on a fait pour l'exercice sur le langage $\{a^n b^n \mid n \geq 0\}$, et qu'on verra avec la relation de Myhill-Nerode (HP). Une autre possibilité est d'utiliser le **lemme de l'étoile**.

Essentiellement, un langage est reconnaissable si et seulement si on peut décider l'appartenance avec une complexité spatiale en $O(1)$.

IV - Algorithme de Berry-Sethi

Cette partie présente un algorithme à connaître pour construire un automate reconnaissant le langage associé à une expression régulière donnée, appelé automate de Glushkov. Cette méthode, utilisant l'algorithme de Berry-Sethi, est moins intuitive que celle de l'automate de Thompson que nous avons vu précédemment, mais elle aura l'avantage de fournir directement un automate sans ϵ -transition (non déterministe).

IV.1 - Langages locaux

Pour appliquer cet algorithme, il faut commencer par un peu de théorie.

Définition 19

Soit L un langage sur un alphabet Σ . On définit les ensembles suivants :

- | | |
|--|--|
| • $P(L) = \{a \in \Sigma \mid a\Sigma^* \cap L \neq \emptyset\}$ | ensemble des premières lettres des mots de L |
| • $S(L) = \{a \in \Sigma \mid \Sigma^* a \cap L \neq \emptyset\}$ | ensemble des dernières lettres des mots de L |
| • $F(L) = \{u \in \Sigma^2 \mid \Sigma^* u \Sigma^* \cap L \neq \emptyset\}$ | ensemble des facteurs de longueur 2 des mots de L |
| • $N(L) = \Sigma^2 \setminus F(L)$ | ensemble des facteurs interdits de longueur 2 pour L |

Remarque :

- La donnée de $F(L)$ détermine entièrement $N(L)$ et inversement.

Proposition 9

Pour tout langage L sur un alphabet Σ , on a :

$$L \setminus \{\epsilon\} \subseteq$$

Remarque :

- L'inclusion ci-dessus signifie simplement qu'un mot non vide ne peut appartenir à L que si sa première lettre est dans $P(L)$, sa dernière lettre dans $S(L)$ et aucun de ses facteurs de longueur 2 dans $N(L)$.

Démonstration. Sur papier !

□

Remarque :

- Le mot vide doit nécessairement être traité à part (il ne commence pas par une lettre de P).
- On n'a pas l'inclusion réciproque en général !

Langage local :

Un langage L sur Σ est dit **local** si :

$$L \setminus \{\epsilon\} = (P(L)\Sigma^* \cap \Sigma^*S(L)) \setminus (\Sigma^*N(L)\Sigma^*)$$

Remarques :

- Un langage local est essentiellement un langage pour lequel l'appartenance peut être décidée en utilisant uniquement une « fenêtre glissante » de largeur 2.
- Les langages locaux sont rationnels, mais ce n'est pas évident à partir de la définition.

Exercice :

Pour chacun des langages suivants, donner $P(L)$, $S(L)$, $F(L)$ et déterminer si L est local.

- | | | |
|-----------------------------|------------------------------------|---------------------------------|
| 1. $L = \mathcal{L}(abbab)$ | 3. $L = \mathcal{L}((ab)^*)$ | 5. $L = \mathcal{L}(a^*(ab)^*)$ |
| 2. $L = \mathcal{L}(a^*)$ | 4. $L = \mathcal{L}((ab)^*(cd)^*)$ | 6. $L = \mathcal{L}((ab)^*a^*)$ |

À faire sur feuille.

Nous allons maintenant énoncer quelques propriétés de stabilité sur les langages locaux, sans les démontrer. De manière générale, les langages locaux et les expressions régulières linéaires (qu'on verra juste après) ne sont pas un objectif central du programme mais simplement « introduits dans la seule perspective de la construction de l'automate de Glushkov associé à une expression régulière par l'algorithme de Berry-Sethi » (qui, elle, est à connaître!).

Soit L, L' deux langages locaux non vides sur Σ . On a :

Étoile de Kleene :

- L^* est local.
- $P(L^*) = P(L)$ et $S(L^*) = S(L)$.
- $F(L^*) = F(L) \cup S(L) \cdot P(L)$.

Union :

- $P(L \cup L') = P(L) \cup P(L')$;
- $S(L \cup L') = S(L) \cup S(L')$;
- $F(L \cup L') = F(L) \cup F(L')$;
- $\Delta N(L \cup L') = N(L) \cap N(L')$.

Concaténation :

- $P(L \cdot L') = \begin{cases} P(L) & \text{si } \epsilon \notin L \\ P(L) \cup P(L') & \text{sinon} \end{cases}$;
- $S(L \cdot L') = \begin{cases} S(L') & \text{si } \epsilon \notin L' \\ S(L) \cup S(L') & \text{sinon} \end{cases}$;
- $F(L \cdot L') = F(L) \cup F(L') \cup S(L) \cdot P(L')$;

Démonstration. Laissée à la discrétion du lecteur. □

Remarque :

- Si L ou L' est vide, alors $L \cdot L' = \emptyset$ (qui est bien sûr local) mais on a $P(L \cdot L') = S(L \cdot L') = F(L \cdot L') = \emptyset$. Toutes les autres formules fonctionnent sans l'hypothèse "non vide".

En revanche, comme le montre l'exercice précédent, ni la concaténation ni l'union de deux langages locaux n'est local en général. On peut cependant retrouver ce résultat si on considère des langages d'alphabets distincts.

Si L et L' sont deux langages locaux sur Σ et Σ' respectivement, avec $\Sigma \cap \Sigma' = \emptyset$. On a :

- $L \cup L'$ est local;
- $L \cdot L'$ est local;

Démonstration. Laissée à la discrétion du lecteur. □

Remarque :

- Les formules donnant P , S et N sont bien valables même si les alphabets ne sont pas distincts (et même d'ailleurs si L et L' ne sont pas locaux). On n'en a besoin que pour garder le caractère local de l'union et de la concaténation (en général).

L'intersection de deux langages locaux est locale (sans condition sur les alphabets).

□

Démonstration. Exercice.

De toutes ces propriétés sur P , S et L , on déduit un **algorithme** permettant de calculer facilement les ensembles P , S et L associés à une expression linéaire.

Il suffit d'appliquer récursivement les formules données (avec les bons cas de base)!

Nous implémenterons cet algorithme dans le TP sur l'algorithme de Berry-Sethi.

IV.2 - Expressions régulières linéaires

On a vu que les langages rationnels ne sont pas locaux, en général. Cependant, ils le deviennent si on restreint toutes les opérations d'union et de concaténation à ne porter que sur des langages d'alphabets disjoints.

On va donc introduire une classe restreinte d'expressions régulières formalisant cette restriction.

Une expression régulière est dite **linéaire** si

Exemples :

- Les expressions régulières $(ab)^*c|de$ et $((ab)^*c)^*$ sont linéaires.
- Les expressions régulières $(aa)^*$ et $(ab)^*|a$ ne sont pas régulières.

Remarque :

- On peut *linéariser* une expression régulière en attribuant un numéro distinct à chacune des lettres qui y apparaissent : $(ab)^*|a$ devient $(a_1b_2)^*|a_3$. Le langage de la nouvelle expression n'est bien sûr plus le même, mais cette opération jouera un rôle important dans la suite du cours. On parlera de **marquage** des lettres.

Si e est une expression régulière linéaire sur un alphabet Σ , alors le langage de e est local.

□

Démonstration. Sur papier!

Remarque :

- La réciproque du théorème est fautive : une expression comme $a|a^*$ n'est pas linéaire mais dénote le même langage (local) que l'expression linéaire a^* .
On peut même aller plus loin : le langage de l'expression aa^* est local mais n'est dénoté par aucune expression linéaire.
- La preuve de ce théorème fournit un algorithme permettant de calculer un automate reconnaissant une expression régulière linéaire, qui sera une étape de l'algorithme de Berry-Sethi.

IV.3 - Automates locaux



Attention! Cette section est à copier sur feuille depuis le tableau.



Exercice d'application :

Construire un automate local reconnaissant $(a|bc)^*$, et un reconnaissant $(a|b)c^*$.

IV.4 - Automate de Glushkov

Soit L un langage rationnel et e une expression régulière le dénotant. L'algorithme de Berry-Sethi pour construire un **automate de Glushkov** reconnaissant L est le suivant :

ALGORITHME DE BERRY-SETHI (sur une expression régulière e) :

- 1) **On linéarise l'expression e par marquage** pour obtenir une expression e' . On peut par exemple remplacer la i -ème occurrence de $x \in \Sigma$ dans e par (x, i) (que l'on notera x_i). On change donc d'alphabet, en remplaçant Σ par une partie (finie) de $\Sigma \times \mathbb{N}$.

On note ϕ l'application d'« oubli des marques » qui remplace chaque lettre (x, i) d'un mot sur $(\Sigma \times \mathbb{N})^*$ par la lettre $x \in \Sigma$ correspondante. On a donc $\mathcal{L}(e) = \phi(\mathcal{L}(e'))$.

- 2) Le langage associé à cette expression étant local, **on construit un automate local standard $\mathcal{A}'$ reconnaissant $\mathcal{L}(e')$** (sans oublier de déterminer si $\epsilon \in \mathcal{L}(e')$ pour choisir si l'état initial est acceptant).

- 3) **On efface les marques sur les transitions de $\mathcal{A}'$** pour obtenir un automate (*a priori* non déterministe) $\mathcal{A}$, appelé **automate de Glushkov**. (C'est à dire qu'on remplace dans $\mathcal{A}$ toutes les transitions de la forme (x, i) par une transition étiquetée par x). Autrement dit, $\mathcal{A}$ a les mêmes états que $\mathcal{A}'$ (y compris l'état initial et les états acceptants), et $q \xrightarrow{x} q'$ dans $\mathcal{A}$ si et seulement si $q \xrightarrow{x_i} q'$ (pour un i quelconque) dans $\mathcal{A}'$.

On a alors (clairement) $\mathcal{L}(\mathcal{A}) = \phi(\mathcal{L}(\mathcal{A}'))$, autrement dit $L(\mathcal{A}) = \mathcal{L}(e) = L$.

Remarques :

- L'automate de Glushkov associé à une expression e possède $|e| + 1$ états, où $|e|$ désigne le nombre de lettres (non nécessairement distinctes) de l'expression e . Il est raisonnablement simple, mais il n'est pas en général minimal. On va donner un contre-exemple juste en dessous.
- On peut bien sûr déterminer l'automate de Glushkov si on le souhaite. On risque cependant d'obtenir un nombre d'états de l'ordre de $2^{|e|}$.

Exemple : Construisons l'automate de Glushkov de l'expression $e = (ab|a)^*c^*$.

Sur papier !

En donner un automate minimal (sans preuve).

V - Automate minimal (HP)

Définition 22

Langages résiduels :

Soient $\mathcal{L}$ un langage sur Σ et $u \in \Sigma^*$. Le **résiduel** (ou **quotient gauche**) de $\mathcal{L}$ par u est l'ensemble :

$$u^{-1}\mathcal{L} = \{v \in \Sigma^* \mid uv \in \mathcal{L}\}$$

Remarque :

- Intuitivement, $u^{-1}\mathcal{L}$ est l'ensemble des mots qu'il "reste à lire" après u pour former un mot de $\mathcal{L}$, i.e. l'ensemble des transitions qu'il reste à suivre dans un automate déterministe pour arriver dans un état final depuis l'état $\delta(q, u)$. D'où la relation ci-dessous :

Définition 23

Relation de Myhill-Nerode :

Soit $\mathcal{L}$ un langage sur Σ . On définit la relation $\sim_{\mathcal{L}}$ sur Σ^* , appelée **relation de Myhill-Nerode**, par, pour tout $x, y \in \Sigma^*$:

$$x \sim_{\mathcal{L}} y \Leftrightarrow (\forall w \in \Sigma^*, (xw \in \mathcal{L} \Leftrightarrow yw \in \mathcal{L})).$$

Proposition 13

Pour tout $u, v \in \Sigma^*$. On a :

- Si $u \sim_{\mathcal{L}} v$, alors Pour tout $w \in \Sigma^*$, $uw \sim_{\mathcal{L}} vw$.
- $u^{-1}\mathcal{L} = v^{-1}\mathcal{L}$ si et seulement si $u \sim_{\mathcal{L}} v$.
- Les classes d'équivalence de $\sim_{\mathcal{L}}$ sont en bijection avec l'ensemble des résiduels de $\mathcal{L}$.

Théorème de Myhill-Nerode :

Un langage $\mathcal{L}$ est rationnel si et seulement si la relation de Myhill-Nerode associée possède un nombre fini de classes d'équivalences, et donc si et seulement si $\mathcal{L}$ possède un nombre fini de résiduels.

Dans ce cas, ce nombre de classes d'équivalences est le **nombre minimal d'états** d'un automate fini déterministe complet reconnaissant $\mathcal{L}$.

Démonstration. • **Minimalité :**

Supposons $\mathcal{L}$ rationnel, et soit alors $\mathcal{A}$ un automate déterministe complet reconnaissant $\mathcal{L}$. Pour q un état de $\mathcal{A}$, notons $P_q = \{u \in \Sigma^* \mid \delta^*(q_I, u) = q\}$ l'ensemble des mots qui nous amènent en q en partant de l'état initial.

Si u et v sont dans le même P_q , alors $u \sim_{\mathcal{L}} v$. En effet, lire uz ou vz depuis l'état initial nous amènera dans un même état $\delta^*(q, z)$, donc uz et vz sont tous les deux acceptés ou tous les deux rejetés.

Chaque P_q est donc inclus dans l'une des classes d'équivalence, et comme la réunion des P_q est égale à Σ^* (l'automate est complet), il faut au moins un état par classe d'équivalence.

En particulier, un langage rationnel a un nombre fini de classes d'équivalences pour la relation de Myhill-Nerode (puisque'il peut être reconnu par un automate fini).

• **Existence :**

Inversement, soit $\mathcal{L}$ un langage ayant un nombre fini de classes d'équivalences $R_1, \dots, R_n$. On définit un automate déterministe complet reconnaissant $\mathcal{L}$ de la manière suivante :

- il y a un état pour chaque classe d'équivalence (on confondra les états et les classes d'équivalence)
- l'état initial est la classe d'équivalence de ϵ
- un état est final si et seulement si il contient au moins un mot de $\mathcal{L}$ (et donc si et seulement si tous ses mots sont dans $\mathcal{L}$)
- si $u \in R$ et $a \in \Sigma$, alors $\delta(R, a)$ est l'état R' qui contient ua . Il faut vérifier que cette définition ne dépend pas du choix (arbitraire) de u , ce qui est bien le cas car si $u \sim_{\mathcal{L}} v$ alors on a $ua \sim_{\mathcal{L}} va$.

Cet automate reconnaît bien $\mathcal{L}$ et possède exactement un état par classe d'équivalence, ce qui montre que le minorant obtenu plus haut est en fait un minimum. □

Remarques :

- Le théorème n'est pas au programme. Il peut servir à prouver qu'un langage n'est pas rationnel, mais il faut savoir le justifier (et donc connaître sa preuve) pour pouvoir l'utiliser aux concours.
- L'automate déterministe minimal dont on vient de prouver l'existence est appelé **automate des résiduels**. Il est en fait unique à un renommage des états près (il suffit de bien regarder la démonstration), et peut être calculé de manière raisonnablement efficace à partir d'un automate déterministe quelconque reconnaissant le langage. Si vous voulez en savoir plus, regardez les exercices de fin de TD.
- Pour déterminer si deux automates reconnaissent le même langage, le plus efficace est de les minimiser et de déterminer si les automates obtenus sont isomorphes (mais ce n'est pas la seule solution).
- L'automate minimal est complet par construction : il possède au maximum un état non co-accessible (état puits). Si c'est le cas, cet état peut être supprimé pour obtenir un automate déterministe (non complet) possédant un état de moins que l'automate minimal et reconnaissant le même langage.

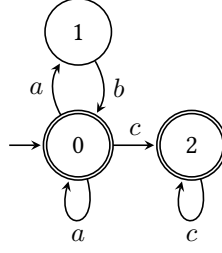
La démonstration fournit également un moyen de décider si un automate est minimal :

Soit $A = (\Sigma, Q, q_0, F, \delta)$ un automate déterministe complet accessible reconnaissant un langage $\mathcal{L}$. Les propositions suivantes sont équivalentes :

- A est minimal (parmi tous les automates déterministes complets reconnaissant $\mathcal{L}$);
- si $q, q' \in Q$, $q \neq q'$, alors il existe $u \in \Sigma^*$ tel que $q.u \in F$ et $q'.u \notin F$ (ou inversement);
- si $q, q' \in Q$, $q \neq q'$, alors $L_q \neq L_{q'}$;
- si $u, v \in \Sigma^*$ sont tels que $u^{-1}\mathcal{L} = v^{-1}\mathcal{L}$, alors $\delta^*(q_0, u) = \delta^*(q_0, v)$.

Annexe : Déroulement de l'algorithme de NcNaughton et Yamada

Considérons l'automate suivant :



On obtient facilement la matrice $R^{(0)}$:

$$R^{(0)} = \begin{pmatrix} \epsilon|a & a & c \\ b & \epsilon & \emptyset \\ \emptyset & \emptyset & \epsilon|c \end{pmatrix}$$

Pour la matrice $R^{(1)}$, une application directe des règles sans aucune simplification donne déjà quelque chose d'assez lourd :

$$R^{(1)} = \begin{pmatrix} (\epsilon|a)|(\epsilon|a)(\epsilon|a)^*(\epsilon|a) & a|(\epsilon|a)(\epsilon|a)^*a & c|(\epsilon|a)(\epsilon|a)^*c \\ b|b(\epsilon|a)^*(\epsilon|a) & \epsilon|b(\epsilon|a)^*a & \emptyset|b(\epsilon|a)^*c \\ \emptyset|\emptyset(\epsilon|a)^*(\epsilon|a) & \emptyset|\emptyset(\epsilon|a)^*a & (\epsilon|c)|\emptyset(\epsilon|a)^*c \end{pmatrix}$$

On peut facilement simplifier :

$$R^{(1)} = \begin{pmatrix} a^* & a^+ & a^*c \\ ba^* & \epsilon|ba^+ & ba^*c \\ \emptyset & \emptyset & \epsilon|c \end{pmatrix}$$

Pour la suite, on effectue dès que possible les simplifications immédiates (élimination des $\emptyset$ en particulier) :

$$\begin{aligned} R^{(2)} &= \begin{pmatrix} a^*|a^+(\epsilon|ba^+)^*ba^* & a^+|a^+(\epsilon|ba^+)^*(\epsilon|ba^+) & a^*c|a^+(\epsilon|ba^+)^*ba^*c \\ ba^*|(\epsilon|ba^+)(\epsilon|ba^+)^*ba^* & \epsilon|ba^+|(\epsilon|ba^+)(\epsilon|ba^+)^*(\epsilon|ba^+) & ba^*c|(\epsilon|ba^+)(\epsilon|ba^+)^*ba^*c \\ \emptyset & \emptyset & \epsilon|c \end{pmatrix} \\ &= \begin{pmatrix} a^*(ba^+)^* & a^*(ba^+)^*a & a^*(ba^+)^*c \\ (ba^+)^*ba^* & (ba^+)^* & (ba^+)^*ba^*c \\ \emptyset & \emptyset & \epsilon|c \end{pmatrix} \end{aligned}$$

L'unique état initial est 0, et les états acceptants sont 0 et 2 : on peut donc se contenter de calculer deux coefficients de la dernière matrice.

$$\begin{aligned} R^{(3)} &= \begin{pmatrix} a^*(ba^+)^* & ? & a^*(ba^+)^*c|a^*(ba^+)^*(\epsilon|c)^*(\epsilon|c) \\ ? & ? & ? \\ ? & ? & ? \end{pmatrix} \\ &= \begin{pmatrix} a^*(ba^+)^* & ? & a^*(ba^+)^*c^* \\ ? & ? & ? \\ ? & ? & ? \end{pmatrix} \end{aligned}$$

Le langage de l'automate est donc décrit par $\boxed{a^*(ba^+)^*|a^*(ba^+)^*c^* = a^*(ba^+)^*c^*}$.

Remarque :

Cette expression équivaut en fait à $(a|ba)^*c^*$.